



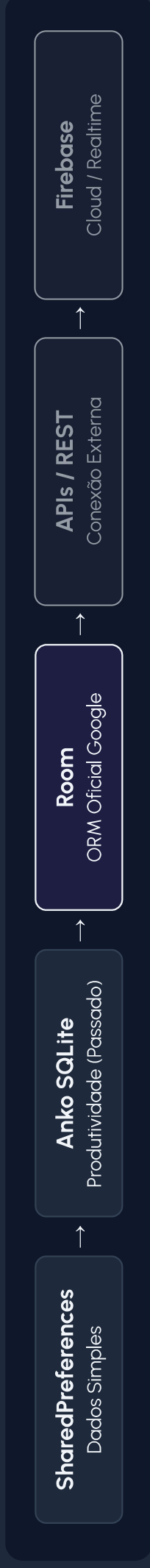
Android **Room** Database

Prof. Dr. Rodrigo Ferreira de Moraes

Instituto Federal do Triângulo Mineiro — IFTM Campus Paracatu

A Jornada do Armazenamento Móvel

Deixamos as soluções procedurais e strings cruas para trás. Bem-vindos à era da Arquitetura!



Meta de Hoje: Garantir que nossos comandos de banco sejam checados durante a compilação e executados sem travar a tela do usuário.

O Problema: ANR e Bugs Silenciosos

1. Erros em Runtime (Bugs Silenciosos)

No SQLite/Anko, escrevemos queries como `"SELECT * FROM Livros"` (com erro de digitação). O app compila normalmente, mas dá Crash quando o usuário clica no botão.

2. Application Not Responding (ANR)

A interface do Android é desenhada na **Main Thread** (60 vezes por segundo). Se fizermos uma busca pesada no banco na mesma thread, a tela congela. O Android exibe a temida janela de "App não está respondendo".

A Solução Dupla:

Para os Bugs: Jetpack Room (ORM) verifica o SQL na compilação.

Para o ANR: Kotlin Coroutines (Processamento paralelo e leve) executa as operações em Background.

Por que o Room se destaca?

Antes do Room, desenvolvedores utilizavam o **SQLiteOpenHelper** (puro) ou outras bibliotecas ORM. O Room revolucionou essa camada:



SQLite Puro (Antigo)

Exigia linhas massivas de código repetitivo (boilerplate). Erros de sintaxe nas queries SQL só quebravam o app em tempo de execução (Runtime), gerando bugs difíceis de rastrear.



O Diferencial do Room

Validação em tempo de compilação: Se você errar o nome de uma coluna ou comando no SQL, o projeto não compila. O Android Studio avisa o erro imediatamente no código.



Integração Reativa

Diferente de soluções antigas, o Room se conecta nativamente com **Kotlin Coroutines (suspend functions)** e fluxos reativos como **Flow** e **LiveData**, atualizando a UI automaticamente.

Entendendo as Funções Suspend

Por padrão, operações de banco de dados (leitura e escrita) são consideradas operações bloqueantes de E/S (I/O). Se executadas na thread principal (Main Thread), elas travam a interface do usuário, resultando no temido erro **ANR (Application Not Responding)**.

A palavra-chave **suspend** do Kotlin resolve isso. Ela sinaliza que a função pode pausar sua execução sem bloquear a thread atual, transferindo o trabalho pesado de forma assíncrona para uma thread de segundo plano (como **Dispatchers.IO**) e retornando o controle quando terminar.

Exemplo de Implementação no DAO

```
@Dao
interface TarefaDao {
    // 0 modificador 'suspend' força o desenvolvedor
    // a chamar esta função de forma assíncrona
    @Insert
    suspend fun inserir(tarefa: Tarefa)

    @Delete
    suspend fun deletar(tarefa: Tarefa)
}
```

Persistência Reativa com Kotlin Flow

Exemplo de Implementação no DAO

```
@Dao
interface TarefaDao {
    // Ao retornar Flow, NÃO usamos a palavra 'suspend'.
    // O Room configura a reatividade automaticamente.
    @Query("SELECT * FROM tarefas_table ORDER BY id DESC")
    fun buscarTodas(): Flow<List<Tarefa>>
}
```

Enquanto uma função **suspend** normal retorna apenas um valor único após concluir (interação do tipo "chame e espere"), o **Flow** representa um fluxo de dados contínuo e em tempo real.

O grande diferencial no Room: Sempre que qualquer modificação acontecer na tabela (um novo insert, update ou delete), o Room detecta a mudança e emite automaticamente a lista atualizada para quem estiver observando o Flow na camada de UI, eliminando a necessidade de consultas manuais repetidas.

Os Três Pilares do Room

A estrutura do Room é estritamente dividida em três componentes que trabalham juntos:

Componente	O que ele faz?	Analogia Prática
Entity (@Entity)	Classe de dados Kotlin que define o esquema da tabela do banco de dados.	A planilha ou tabela física onde as linhas de dados são inseridas.
DAO (@Dao)	Interface contendo os métodos usados para acessar o banco (Inserir, Deletar, Consultar).	O garçom ou intermediário: você pede a ação e ele executa no banco.
Database (@Database)	O ponto central de conexão que estende o RoomDatabase e gerencia as instâncias.	O gerente do restaurante, que abre as portas e garante uma única cozinha funcionando.

1. Configuração de Dependências

```
build.gradle.kts (Module :app)
```

```
plugins {  
    // Adicionar essa linha ao final de puglins  
    alias(libs.plugins.google.devtools.ksp)  
}  
  
dependencies {  
    // Adicionar essa linha ao final de dependencies  
    val room_version = "2.6.1"  
    implementation("androidx.room:room-runtime:$room_version")  
    implementation("androidx.room:room-ktx:$room_version")  
    ksp("androidx.room:room-compiler:$room_version")  
}
```


| 2. Configuração de Dependências

```
build.gradle.kts (Module :Projeto)
```

```
plugins {  
    // Adicionar essa linha ao final de puglins  
    alias(libs.plugins.google.devtools.ksp) apply false  
}
```

3. Configuração de Dependências

libs.version.toml

```
[versions]
// Adicionar essa linha ao final de versions
ksp = "2.0.0-1.0.24"

[plugins]
// Adicionar essa linha ao final de plugins
google-devtools-ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
```

4. Criando a Entidade (Tabela)

Tarefa.kt

```
package com.example.roomsimple

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "tarefas_table")
data class Tarefa(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val descricao: String,
    val estaConcluida: Boolean = false
)
```

 **@Entity** avisa ao Room para mapear essa classe como tabela. O **autoGenerate = true** faz o SQLite criar IDs incrementais automaticamente.

5. Criando o DAO (Ações do CRUD)

TarefaDao.kt

```
package com.example.roomsimple

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow

@Dao
interface TarefaDao {
    @Insert
    suspend fun inserir(tarefa: Tarefa)

    @Update
    suspend fun atualizar(tarefa: Tarefa)

    @Delete
    suspend fun deletar(tarefa: Tarefa)

    @Query("SELECT * FROM tarefas_table ORDER BY id DESC")
    suspend fun buscarTodas(): Flow<Tarefa>

    @Query("DELETE FROM tarefas_table")
    suspend fun apagarTudo()
}
```

 A palavra chave **suspend** obriga essas funções a serem chamadas dentro de Coroutines, garantindo que o banco nunca trave a interface do usuário.

6. Criando o Banco de Dados (Singleton)

AppDatabase.kt

```
package com.example.roomsimple

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(entities = [Tarefa::class], version = 1, exportSchema = false)
abstract class AppDatabase : RoomDatabase() {

    abstract fun tarefaDao(): TarefaDao

    companion object {
        @Volatile
        private var INSTANCE: AppDatabase? = null

        fun getDatabase(context: Context): AppDatabase {
            return INSTANCE ?: synchronized(this) {
                val instance = Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "tarefa_database"
                )
                    .fallbackToDestructiveMigration()
                    .build()
                INSTANCE = instance
            }
        }
    }
}
```

| 7. Layout da Interface de Usuário

res/layout/activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://tools.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp"
    tools:context=".MainActivity">

    <EditText
        android:id="@+id/editTextTarefa"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Digite uma nova tarefa"
        android:inputType="textPersonName"
        android:minHeight="48dp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:layout_marginTop="8dp"
        android:gravity="center">

        <Button
            android:id="@+id/btnSalvar"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Salvar" />

        <Button
            android:id="@+id/btnLimpar"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1" />

    </LinearLayout>
</LinearLayout>
```

```
    android:layout_marginStart="8dp"
    android:text="Limpar Tudo" />
</LinearLayout>

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Tarefas Salvas no SQLite:"
    android:textStyle="bold"
    android:layout_marginTop="16dp"
    android:textSize="16sp" />

<ScrollView
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:layout_marginTop="8dp">

    <TextView
        android:id="@+id/textViewResultados"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Nenhuma tarefa cadastrada."
        android:textSize="14sp" />
    </ScrollView>
</LinearLayout>
```

8. Acoplamento e Execução Final

MainActivity.kt

```
package com.example.roomsamples

import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.LifecycleScope
import kotlinx.coroutines.launch

class MainActivity : AppCompatActivity() {

    private lateinit var database: AppDatabase
    private lateinit var tarefaDao: TarefaDao
    private lateinit var editTextTarefa: EditText
    private lateinit var btnSalvar: Button
    private lateinit var btnLimpar: Button
    private lateinit var textViewResultados: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        database = AppDatabase.getDatabase(this)
        tarefaDao = database.tarefaDao()

        editTextTarefa = findViewById(R.id.editTextTarefa)
        btnSalvar = findViewById(R.id.btnSalvar)
        btnLimpar = findViewById(R.id.btnLimpar)
        textViewResultados = findViewById(R.id.textViewResultados)

        atualizarListaDeTarefas()

        btnSalvar.setOnClickListener {
            val descricao = editTextTarefa.text.toString().trim()
```



```

if (descricao.isEmpty()) {
    val novaTarefa = Tarefa(descricao = descricao)
    lifecycleScope.launch {
        tarefaDao.inserir(novaTarefa)
        editTextTarefa.text.clear()
        Toast.makeText(this@MainActivity, "Salvo com sucesso!", Toast.LENGTH_SHORT).show()
    }
}

btnLimpar.setOnClickListener {
    lifecycleScope.launch {
        tarefaDao.apagarTudo()
    }
}

private fun atualizarListaDeTarefas() {
    lifecycleScope.launch {
        tarefaDao.buscarTodas().collect { lista →
            if (lista.isEmpty()) {
                textViewResultados.text = "Nenhuma tarefa cadastrada."
            } else {
                val stringBuilder = StringBuilder()
                for (tarefa in lista) {
                    stringBuilder.append("💎 [ID: ${tarefa.id}] ${tarefa.descricao}\n\n")
                }
                textViewResultados.text = stringBuilder.toString()
            }
        }
    }
}

```

Desafio Prático: Evoluindo o App

O código atual insere e apaga todas as tarefas de uma vez. Sua missão é evoluir a arquitetura para suportar o gerenciamento individual de estado das tarefas.

Tarefa 1: Marcar como Concluída

Implementar a funcionalidade que permite ao usuário clicar em uma tarefa específica para alternar seu status entre "Pendente" e "Concluída".

- **Passo 1 (DAO):** Crie um método `@Update` para atualizar uma `Tarefa` existente.
- **Passo 2 (UI):** Adicione um `CheckBox` no layout e associe ao clique do item.
- **Passo 3 (Lógica):** Inverta o valor de `estaConcluída` e chame a função de update no `lifecycleScope`.

Tarefa 2: Deleção Individual

Permitir que o usuário apague apenas uma tarefa específica, em vez de limpar o banco inteiro.

- **Passo 1 (DAO):** Crie um método `@Delete` recebendo um objeto `Tarefa`.

- **Passo 2 (UX):** Implemente um botão de "lixeira" ao lado de cada tarefa ou use a ação de "Long Click" (clique longo) no item da lista para disparar a deleção.

Bibliografia e Referências

Aprofunde seus conhecimentos acessando a documentação oficial e os guias de arquitetura recomendados para o ecossistema Android.



Android Developers: Salvar dados em um banco de dados local usando o Room

Guia oficial de treinamento e arquitetura do Android Jetpack.

developer.android.com/training/data-storage/room



Kotlin Documentation: Asynchronous Flow

Documentação principal sobre fluxos reativos em Kotlin Coroutines.

kotlinlang.org/docs/flow.html



Android Dev Summit: Room Coroutines & Flow

Palestras técnicas da equipe de engenharia do Google abordando as melhores práticas do Room.

youtube.com/androiddevelopers